

Project Architecture & Decision Guide

A complete end-to-end walkthrough of the codebase — what every piece does, why it was built that way, and how it all fits together.

Stack	Next.js 16 (App Router) · Hono · Cloudflare Workers · Supabase
Audience	Developer / Technical Owner
Version	1.0 – May 2026

Table of Contents

1	Project Overview & Business Context
2	Tech Stack — Choices & Reasons
3	Repository Structure
4	Backend Deep-Dive (repaiross-api)
4.1	Entry Point & Middleware
4.2	Authentication — /auth
4.3	Jobs — /jobs (core workflow)
4.4	Bookings — /bookings
4.5	Bills — /bills
4.6	Customers & Vehicles
4.7	Leads — /leads
4.8	WhatsApp Webhook — /webhook/whatsapp
4.9	Notifications — /notify
4.10	OTP — /public/otp
4.11	Public Booking — /public/booking
4.12	Staff — /staff
4.13	Dashboard — /dashboard
5	Database Schema & Data Model
6	WhatsApp Integration
7	Frontend — RapidFixFrontendFINAL
7.1	Landing Page
7.2	Booking Flow (Public)
7.3	Admin Panel
8	State Management & API Client
9	Key Design Decisions — Summary Table
10	Data Flow Diagrams
11	Planned Features & Current Gaps

1. Project Overview & Business Context

What is RapidFix and why does this system exist?

RapidFix is an Indian automotive service business (cars & bikes) that operates a physical workshop. The digital product has two distinct goals:

- **Customer-facing:** A marketing website that lets visitors learn about services, book a slot online, and stay informed via WhatsApp.
- **Operations-facing:** An admin panel that lets the owner and mechanics manage every job from the moment a vehicle arrives until it is delivered and paid for.

The Business Problems Being Solved

Before this system, the workshop relied on phone calls, WhatsApp messages and paper records. Three problems drove this build:

- **Missed leads:** Enquiries on WhatsApp that came outside working hours were forgotten. The system now captures every inbound WA message as a lead automatically.
- **No customer visibility:** Customers had to call to find out where their car was. The system now sends WhatsApp notifications at every status change.
- **Paper billing:** Invoices were handwritten. The system now generates itemised digital bills with a public link the customer can open on their phone.

Scope at a Glance

Module	What it covers
Landing page	Brand, services, pricing, booking CTA, lead capture popup
Online booking flow	Multi-step wizard: vehicle → service → date → contact → checkout
Admin — Jobs	Full job lifecycle: received → diagnosed → in_progress → ready → delivered
Admin — Bookings	Slot management, confirmation, WhatsApp confirmations
Admin — Bills	Itemised invoices (labour + parts), payment tracking, public bill link
Admin — Customers	Customer DB, vehicle history, per-customer WhatsApp thread
Admin — Leads	WhatsApp-sourced enquiries, assignment, conversion to customer
Admin — Inbox	Unified WhatsApp conversation view per lead/customer
Admin — Dashboard	Revenue KPIs, open job counts, recent activity feed

2. Tech Stack — Choices & Reasons

Every tool in the stack was picked deliberately.

Frontend

Next.js 16 (App Router)	React meta-framework chosen for its file-based routing, React Server Components, and image optimisation. Version 16 with App Router means layouts, server-side rendering, and API routes are co-located with UI code.
TypeScript	End-to-end type safety across the frontend. The same type definitions in <code>src/lib/types.ts</code> describe both what the API returns and what the UI consumes, catching shape mismatches at compile time rather than at runtime.
Tailwind CSS v4	Utility-first CSS. There is no custom CSS file to maintain; all styling is inline class names. TW v4 ships a JIT-only compiler with zero-config PostCSS setup.
Zustand + persist	Lightweight global state for the multi-step booking wizard. <code>persist</code> middleware writes wizard state to <code>localStorage</code> so the user can resume a partially-filled form across page refreshes.
GSAP / Framer Motion	Animation libraries used exclusively on the landing page for hero scroll effects and service carousels. Not used inside the admin panel to keep bundle size small.
Lucide React	Icon set. Only icons actually used are tree-shaken into the bundle.

Backend

Hono	Minimal, edge-native TypeScript web framework. It runs on Cloudflare Workers natively — no Node.js runtime needed in production. Hono's router is the fastest available in the Workers environment.
Cloudflare Workers	The backend runs as a serverless edge function deployed globally on Cloudflare's network. No servers to manage, no cold starts on warm Workers, and automatic global distribution. Scales to zero cost on quiet days.
Supabase (PostgreSQL)	Managed Postgres with a JavaScript client SDK. Supabase was chosen over raw Postgres because it provides Auth (used for staff login via email+password), Row Level Security, and a generous free tier. The service-role key is used server-side so RLS is bypassed in favour of application-level auth.
Custom JWT (Hono/jwt)	After Supabase Auth validates staff credentials, a separate short-lived JWT (7 days) is issued using the application's own <code>JWT_SECRET</code> . This decouples the admin panel from Supabase's Auth tokens and embeds the <code>role/location_id</code> claims that every route needs.
WhatsApp Business API (Meta Graph)	Notifications (job status, booking confirmations, bills) are delivered via WhatsApp templates. This was chosen over SMS because Indian customers are already active on WhatsApp and open rates are much higher than SMS. Template messages are pre-approved by Meta.

Design Decision: Why Cloudflare Workers instead of a traditional Node.js server or Vercel Edge Functions? Workers have no cold-start penalty for HTTP workloads, the free plan covers millions of requests/month, and wrangler deploy gives a one-command CI/CD pipeline. The trade-off is that Workers cannot run native Node

modules — hence hono/jwt instead of jsonwebtoken (which uses Node crypto), and the WhatsApp calls use the fetch API directly.

Design Decision: Why Supabase as the database instead of Cloudflare D1? D1 is still maturing. Supabase brings Postgres (full SQL), a rich JS client, built-in Auth, and a web dashboard for inspecting data — all valuable during rapid iteration. D1 can be considered for cost optimisation later.

3. Repository Structure

Two sub-projects live in one git repository.

The repo root contains two independent projects:

Path	What it is
RapidFixFrontendFINAL/	The Next.js application — landing page + admin panel
repaiross-api/	The Hono + Cloudflare Workers API backend
repaiross-api/src/index.ts	Entry point: instantiates Hono app, registers all route modules, exports the Worker default
repaiross-api/src/routes/	One file per domain: auth, jobs, bookings, bills, customers, vehicles, leads, staff, dashboard, webhook, notify, Public, otp
repaiross-api/src/middleware/auth.ts	JWT verification middleware + role guards (requireAuth, requireOwner)
repaiross-api/src/type/lib/	db.ts (Supabase client factory), whatsapp.ts (WA send helpers)
repaiross-api/wrangler.jsonc	Cloudflare Workers deployment config — name, compatibility, var bindings
RapidFixFrontendFINAL/src/app/	Next.js App Router pages: /, /admin, /booking, /checkout, /about, /contact, /mechanic-near-me-in-[city]
RapidFixFrontendFINAL/src/components/	Landing page components (AnimatedServices, HeroSlideshow, etc.) + admin/ sub-tree
RapidFixFrontendFINAL/src/components/admin/	Admin shell (Admin.tsx, Sidebar, Topbar) and all admin pages (Dashboard, Jobs, Bookings, Bills, Customers, Leads, Inbox, Settings)
RapidFixFrontendFINAL/src/lib/	api.ts (typed API client), types.ts (shared TypeScript types), constants.ts (design tokens + status configs), utils.ts
RapidFixFrontendFINAL/src/store/	useBookingStore.ts — Zustand store for the public booking wizard

Design Decision: Why are the two projects in a single repo? They share no build tooling or dependencies — each has its own package.json. The single repo is a convenience for the solo developer: one git clone, one place to track issues, and easy cross-reference when the frontend API types need to match the backend routes.

4. Backend Deep-Dive (repaROSS-api)

Hono + Cloudflare Workers + Supabase

4.1 Entry Point & Middleware

The file `src/index.ts` is the entire Worker entry point. It:

- Creates a single Hono app instance typed with the `Env` interface (all secrets/vars injected by Cloudflare at runtime).
- Attaches global middleware: `logger()` for structured logs and `cors()` with wildcard origin — the frontend and mobile apps need to call from different domains.
- Mounts every route module under its own path prefix (e.g. `app.route('/jobs', jobs)`).
- The `Env` interface defines the secrets: `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`, `JWT_SECRET`, `WA_NUMBER_ID`, `WA_ACCESS_TOKEN`, `WA_VERIFY_TOKEN`, `WA_APP_SECRET`, `OWNER_WA_NUMBER`.

Design Decision: Secrets are stored as Cloudflare Worker secrets (encrypted at rest, never in `wrangler.jsonc`). The only var in `wrangler.jsonc` is the non-secret `SUPABASE_URL`.

4.2 Authentication — /auth

Two-step login process:

- **Step 1 — Supabase Auth:** `POST /auth/login` delegates the email+password check to Supabase Auth's `signInWithPassword`. If Supabase rejects it, a 401 is returned immediately.
- **Step 2 — Staff record lookup:** If Supabase approves, the backend fetches the matching row from the `staff` table using the Supabase user `UUID` stored in `auth_user_id`. This row must be `active: true` — inactive staff cannot log in even with valid credentials.
- **Step 3 — Issue custom JWT:** A new JWT is signed with the app's own `JWT_SECRET`. Its payload contains `sub` (staff `UUID`), `name`, `role` (`owner/admin/mechanic`), and `location_id`. TTL is 7 days.

The `requireAuth` middleware (`src/middleware/auth.ts`) validates this token on every protected route. The `requireOwner` middleware additionally enforces that only `owner` or `admin` roles can proceed. Mechanics can only access their own jobs.

Design Decision: Why issue a custom JWT rather than using the Supabase JWT? The Supabase JWT does not carry the `role` or `location_id` claims that every route needs. Embedding them in the token avoids an extra DB lookup on every request.

4.3 Jobs — /jobs (core workflow)

A job is the central entity in the system. It represents one vehicle visit.

Method	Endpoint	Auth	Description
GET	/jobs	requireAuth	List jobs with filters: <code>status</code> , <code>mechanic_id</code> , <code>page</code> , <code>limit</code> . Mechanics only see their own jobs.
POST	/jobs	requireOwner	Create a job. Sets <code>status='received'</code> , logs first status history entry. If <code>booking_id</code> provided, marks booking as 'converted'.

GET	/jobs/:id	requireAuth	Full job detail: job + customer + vehicle + mechanic + status history + notes + bill summary.
PATCH	/jobs/:id/status	requireAuth	Advance status. Appends to job_status_history. Sends WhatsApp template if customer opted in.
PATCH	/jobs/:id	requireOwner	Edit job fields: mechanic, estimated completion, odometers, service description, vehicle.
POST	/jobs/:id/notes	requireAuth	Append an internal note. Mechanic-scoped: only their own job.

Job Status Lifecycle

Statuses are strictly ordered and validated server-side:

received	Vehicle has arrived. Created automatically when a job is opened.
diagnosed	Mechanic has assessed the vehicle and knows what needs doing.
in_progress	Repair work is actively underway.
ready	Vehicle is repaired and ready for customer pickup. WA notification sent: 'Ready for pickup!'
delivered	Customer has collected the vehicle. Final state — job disappears from the active dashboard.

Design Decision: The status transition logic lives entirely in the API, not the frontend. The frontend can only send a valid status value; the API validates it against the allowed list. This prevents the frontend from accidentally setting a non-existent status.

4.4 Bookings — /bookings

A booking is an advance slot reservation. It becomes a job when the vehicle arrives.

Method	Endpoint	Auth	Description
GET	/bookings	requireAuth	List bookings with filters: status, date (IST-aware), location_id.
POST	/bookings	requireOwner	Create booking. Source: online/whatsapp/phone/walkin.
PATCH	/bookings/:id	requireOwner	Update slot, notes, vehicle, or status.
POST	/bookings/:id/confirm	requireOwner	Mark confirmed + send WhatsApp confirmation template with slot date/time.
DELETE	/bookings/:id	requireOwner	Hard delete. Returns 404 if not found.

Note: Date filtering compares in IST (+05:30) to prevent UTC midnight from bleeding into adjacent Indian calendar days — a common bug in systems built without timezone awareness.

4.5 Bills — /bills

Bills are itemised invoices attached to a job. Each bill has a public_token that generates a customer-facing view.

Method	Endpoint	Auth	Description
GET	/bills/b/:token	Public	Returns full bill data for customer-facing view (no auth required).
GET	/bills	requireOwner	Paginated list with status filter.
POST	/bills	requireOwner	Create draft bill from job_id. Optionally include line items and discount.
GET	/bills/:id	requireOwner	Full bill with items, customer, vehicle, location.
PATCH	/bills/:id	requireOwner	Replace all items, update discount/notes. Totals are auto-recalculated.
POST	/bills/:id/send	requireOwner	Mark as sent, deliver via WhatsApp / email / both.
PATCH	/bills/:id/payment	requireOwner	Record payment. Auto-calculates status: paid / partial / unpaid.

Bill Calculation Logic

The `recalcTotals()` helper recalculates the bill any time items or discount change:

- `subtotal` = sum of all `bill_items.total` rows
- `total` = `max(0, subtotal - discount)`
- `amount_due` = `total - amount_paid`
- Bill status is computed from `amount_paid`: paid (full), partial (some), unpaid (zero).

Design Decision: Each bill gets a `uuid-based public_token` at creation time. This token is the only thing needed to view the bill — no login required. This lets the customer open the bill on their phone from a WhatsApp link without needing to create an account.

4.6 Customers & Vehicles

Customers and vehicles are separate entities linked by `customer_id`. A customer can have multiple vehicles.

Method	Endpoint	Auth	Description
GET	/customers	requireAuth	Search by name or phone. Paginated.
POST	/customers	requireOwner	Create customer. Phone must be unique (23505 = duplicate).
GET	/customers/lookup	requireAuth	Exact phone lookup — used before creating a job to avoid duplicates.
GET	/customers/:id	requireAuth	Customer + all vehicles + last 10 jobs.
PATCH	/customers/:id	requireOwner	Update name, email, whatsapp_opt_in. Phone is immutable.
GET	/customers/:id/messages	requireOwner	Full WhatsApp conversation thread.
POST	/customers/:id/messages	requireOwner	Send outbound WA free-text message.
POST	/vehicles	requireOwner	Create vehicle. type must be 'car' or 'bike'.
DELETE	/vehicles/:id	requireOwner	Blocked if vehicle has active jobs.

Design Decision: Phone number is the natural key for customers. The `/customers/lookup?phone=` endpoint exists specifically for the 'create job' flow: the admin types a phone number and the UI auto-fills customer

details if they already exist, preventing duplicate records.

4.7 Leads — /leads

A lead is an unqualified contact — either a website popup submission or an inbound WhatsApp message from an unknown number. Leads can be converted to customers.

Method	Endpoint	Auth	Description
POST	/leads/capture	Public	Website popup submit. Validates Indian 10-digit mobile, deduplicates within 24 hrs, notifies owner via WA.
GET	/leads	requireOwner	List leads with status/assignment filters.
PATCH	/leads/:id	requireOwner	Update status (new/follow_up/converted/spam) or assign to staff.
POST	/leads/:id/convert	requireOwner	Promote lead to customer. Re-links all messages to new customer record.
GET	/leads/:id/messages	requireOwner	Conversation thread for this lead.
POST	/leads/:id/messages	requireOwner	Reply to lead via WhatsApp.

4.8 WhatsApp Webhook — /webhook/whatsapp

Meta sends all inbound WhatsApp events (messages + delivery status updates) to this endpoint via HTTP POST.

Security:

- GET request: Meta's subscription verification — checks `hub.verify_token` against the stored `WA_VERIFY_TOKEN` secret and returns the `hub.challenge`.
- POST request: HMAC-SHA256 signature verification using `WA_APP_SECRET`. Any request with an invalid signature is rejected 403.

Inbound Message Processing:

- Deduplication: each message has a unique `wa_message_id` stored in the messages table. The same message ID is never processed twice.
- Known customer: message is stored against `customer_id`.
- Unknown number: a lead is upserted (conflict on `wa_id`), the owner is notified, and a welcome template is sent to the new contact.
- Delivery receipts (status events from Meta) update the `messages.status` column.

Design Decision: The webhook intentionally does not block on the owner notification WA send — it's fire-and-forget (`.catch(console.error)`) so a failed notification never prevents message storage.

4.9 Notifications — /notify

/notify/blast (POST, requireOwner): Sends a WhatsApp free-text message to all opted-in customers, or to a specific subset via `customer_ids[]`. Messages are sent sequentially (not parallel) to avoid Meta rate-limit spikes. Every send — success or failure — is logged to the messages table.

Note: This is a bulk marketing feature. It will only work for customers who have `whatsapp_opt_in = true`. Opt-out handling should be added before large-scale use.

4.10 OTP — /public/otp

Provides phone-number verification for the public booking flow:

- **POST /public/otp/send:** Generates a 6-digit code, stores it in the `otps` table with a 10-minute expiry, and sends it via a Meta WhatsApp authentication template (`otp_verify`). Invalidates any previous unused OTPs for the same number.
- **POST /public/otp/verify:** Checks code, expiry, and used flag. Marks the OTP as used on success.

4.11 Public Booking — /public/booking

The unauthenticated endpoint called by the frontend booking wizard. It handles the full customer + vehicle + booking creation in one call:

- **Customer:** If `customerId` is not supplied, a new customer is created from the contact field. Phone or email is accepted.
- **Vehicle:** The system checks for an existing matching vehicle (same make + model) to avoid duplicates. If not found, a new one is created.
- **Booking:** A pending booking is created with `source='online'`.
- **Owner notification:** A WA message is sent to `OWNER_WA_NUMBER` with full booking details.

Design Decision: The public booking endpoint does not require authentication because it is the customer-facing side. However, it performs all operations via the service-role Supabase client (not RLS), so it is important that it never exposes or modifies staff/admin data.

4.12 Staff — /staff

Staff records represent workshop employees. Every staff member is scoped to a location.

Method	Endpoint	Auth	Description
GET	/staff	requireAuth	List active staff at current location. Owner can also see inactive staff.
GET	/staff/:id	requireOwner	Single staff member (location-scoped).
POST	/staff	requireOwner	Create staff. Roles: owner / admin / mechanic.
PATCH	/staff/:id	requireOwner	Update staff. Cannot modify staff from other locations.

Note: Staff records exist independently of Supabase Auth users. A new staff row does NOT automatically create a Supabase Auth user. The `auth_user_id` link must be set manually or via a future onboarding flow.

4.13 Dashboard — /dashboard

Single GET endpoint that returns a dashboard snapshot in one call. It runs four Supabase queries in parallel:

- Active jobs grouped by status (received / diagnosed / in_progress / ready).
- New lead count for today.
- Revenue from paid/partial bills — segmented into today / this week / this month.
- Last 10 job status history entries with staff name + customer + vehicle.

Design Decision: Revenue is calculated from `bills.amount_paid` (cash actually received), not `bills.total` (amount invoiced). This is intentional — it reflects real cash flow rather than outstanding receivables.

5. Database Schema & Data Model

All tables live in a single Supabase (PostgreSQL) project.

Core entity relationships:

locations	A physical workshop branch. Staff, jobs, bookings, and bills are all scoped to a location.
staff	Workshop employees. Linked to Supabase Auth via auth_user_id. Roles: owner, admin, mechanic.
customers	People who have their vehicle serviced. Phone is unique (natural key). Has whatsapp_opt_in flag.
vehicles	Belongs to a customer. type is 'car' or 'bike'. Has make, model, registration, color, year.
bookings	Advance slot reservation. status: pending → confirmed → arrived / cancelled / converted. source: online/whatsapp/phone/walkin.
jobs	One vehicle visit. Links to customer, vehicle, mechanic (staff), booking, location. Status flows through 5 stages.
job_status_history	Append-only log of every status change on a job. Records who changed it, when, any note, and whether a WA message was sent.
job_notes	Internal staff notes on a job. Not visible to the customer.
bills	Itemised invoice for a job. Has public_token for customer-facing view. Tracks subtotal, discount, total, amount_paid.
bill_items	Line items on a bill. category: labour / parts / consumables / other. total = quantity × unit_price (computed by trigger or app).
leads	Unqualified contacts. Source: website_popup or WhatsApp inbound. Has wa_id for WA-sourced leads. Can be converted to customer.
messages	Unified WhatsApp message log. direction: inbound/outbound. Linked to either lead_id or customer_id. Stores wa_message_id for deduplication.
otps	One-time passwords for phone verification. Has phone, code, expires_at, used flag.

Key Relationships Diagram (text form)

locations → staff (1:N)
locations → jobs (1:N)
customers → vehicles (1:N)
customers → jobs (1:N)
vehicles → jobs (1:N)
jobs → job_status_history (1:N, append-only)
jobs → job_notes (1:N)
jobs → bills (1:1)
bills → bill_items (1:N)
leads → messages (1:N)

customers → messages (1:N)

bookings → jobs (1:1, optional – walkin jobs have no booking)

Design Decision: The job_status_history table is append-only by convention (there are no UPDATE or DELETE operations on it). This creates an auditable trail of every status change. The wa_sent flag on each history row tells you whether a notification was actually delivered for that transition.

6. WhatsApp Integration

The backbone of customer communication.

WhatsApp is used for four distinct purposes:

Job status notifications	Template: <code>job_status_v2</code> . Sent when a job status changes (diagnosed, in_progress, ready, delivered). Variables: customer name, vehicle label, status description.
Booking confirmation	Template: <code>booking_confirmed_v2</code> . Sent when a booking is explicitly confirmed. Variables: customer name, slot date/time.
Bill delivery	Template: <code>bill_ready</code> . Sent with a public bill link. Variables: customer name, total amount, bill URL.
OTP verification	Template: <code>otp_verify</code> (authentication category). Sent with a 6-digit code during public booking phone verification.
Owner notifications	Free-text messages to <code>OWNER_WA_NUMBER</code> when new leads arrive (both from website popup and inbound WA). Not a template — uses <code>waSendText</code> .
Bulk marketing blast	Free-text messages to all opted-in customers via <code>/notify/blast</code> . Planned feature.

How Template Sending Works

The `waSendTemplate()` helper (`src/type/lib/whatsapp.ts`) posts to Meta Graph API v25.0:

- Endpoint: `https://graph.facebook.com/v25.0/{WA_NUMBER_ID}/messages`
- Auth header: `Bearer {WA_ACCESS_TOKEN}`
- Payload includes: `messaging_product: 'whatsapp'`, `type: 'template'`, template name, language code, and body parameters.
- Template variables are positional (`{{1}}`, `{{2}}`, ...) and must match the approved template's parameter count exactly.

OTP templates are authentication-category and require an extra button component in the payload:

- Body component with `type: 'text'` parameter containing the code.
- Button component with `sub_type: 'url'`, `index: '0'` also containing the code — Meta mandates this for authentication templates.

Design Decision: WhatsApp notifications are opt-in (`whatsapp_opt_in` flag on customers). The system never sends to a customer who hasn't opted in. This flag defaults to true for new customers because opting in is presented as a feature (track your vehicle) during onboarding.

Note: Template names (`job_status_v2`, `booking_confirmed_v2`, `bill_ready`, `otp_verify`, `rapidfix_welcome`) must be pre-approved by Meta before they can be used. If a template send fails, the API logs the error but does NOT fail the primary operation (status update, booking confirmation, etc.).

7. Frontend — RapidFixFrontendFINAL

Next.js 16 App Router · TypeScript · Tailwind CSS

7.1 Landing Page

The landing page is the public face of RapidFix. It is a server-rendered Next.js page (/src/app/page.tsx) with the following sections (in order):

- **Hero / ScrollExpandMedia:** A full-viewport scroll-driven animation that expands the hero image as the user scrolls down. Uses GSAP ScrollTrigger.
- **Guarantees Bar:** Three value propositions: 30-day warranty, same-day delivery, transparent pricing.
- **Animated Services:** Service cards with hover animations (Framer Motion). Loaded lazily with next/dynamic to avoid blocking the initial paint.
- **Animated Workflow:** Step-by-step 'how it works' section with scroll-triggered animations.
- **Recommendation Strip:** Social proof / Google-review style strip.
- **Brands Strip:** Car brand logos RapidFix services.
- **Animated Testimonials:** Customer review carousel.
- **Statistics Strip:** Key numbers (vehicles serviced, etc.).
- **Service Flow:** Visual representation of the booking-to-delivery flow.
- **LocateUs:** Embedded MapLibre GL map showing workshop location.
- **FAQSection:** Accordion FAQ.
- **StickyWhatsApp:** Fixed bottom-right WhatsApp button that links to the workshop's WA number.
- **LeadPopup:** Timed popup asking for phone number, submitted to POST /leads/capture.
- **Footer:** Links, social icons, address.

Design Decision: Heavy animation libraries (GSAP, Framer Motion) are isolated to the landing page. The admin panel uses zero animation libraries — it intentionally uses plain CSS transitions to keep the admin bundle lean.

7.2 Public Booking Flow

The booking flow is a multi-step wizard at /booking → /checkout. State is managed by the Zustand store (useBookingStore) with localStorage persistence.

Wizard Steps:

Step 1 — Vehicle Type	User selects Car or Bike.
Step 2 — Engine/CC	Car: petrol/diesel/CNG/EV. Bike: CC range (0-150, 150-250, 250-400, 450-650).
Step 3 — Brand	Brand picker filtered by vehicle type.
Step 4 — Model	Model picker filtered by brand.
Step 5 — Service Type	Services available for this vehicle/CC combination (e.g. General Service, General Service with Engine Oil, Jumpstart, Puncture).
Step 6 — Date	Calendar date picker (react-aria-components).

Checkout page	Contact, address, problem description, payment method (pay now / pay later). On submit, calls POST /public/booking via the server action.
----------------------	---

Pricing is computed entirely client-side by `getEstimate()` in the Zustand store, using a lookup table of `service × CC → price`. No API call is needed for the estimate.

Design Decision: The booking wizard uses Zustand persist (localStorage) so if the user closes the tab mid-way and comes back, they continue from where they left off. The store resets only on explicit submission or reset.

7.3 Admin Panel

The admin panel lives at /admin. It is a single-page application rendered client-side ("use client") within the Next.js shell.

Auth Guard

The /admin/page.tsx uses the AuthContext. On load, it checks localStorage for a token, calls /auth/me to validate it, and either renders the LoginForm (if no valid token) or the Admin shell (if valid). There is no server-side session — auth is entirely client-side JWT.

Admin Shell Structure

<code>Admin.tsx</code>	Root component. Renders Sidebar + Topbar + the active page component. Passes staff payload as prop so every page knows the user's role.
<code>Sidebar.tsx</code>	Navigation sidebar. Uses NAV constant from constants.ts. ownerOnly nav items are hidden for mechanics.
<code>Topbar.tsx</code>	Header bar with page title and logout button.
<code>pages/Dashboard.tsx</code>	KPI cards (open jobs by status, revenue today/week/month, new leads today) + today's bookings table + recent activity feed. Loaded from /dashboard.
<code>pages/Jobs.tsx</code>	Filterable table of jobs. Inline status changer (JobStatusBadge) that calls PATCH /jobs/:id/status. New Job modal (NewJobModal) for creating jobs.
<code>pages/Bookings.tsx</code>	Bookings table with date filter. Can confirm or cancel. Confirm triggers WA template.
<code>pages/Bills.tsx</code>	Bill list + draft creation flow + payment recording. Shows bill status (draft/sent/paid/partial/unpaid).
<code>pages/Customers.tsx</code>	Searchable customer list. Drill into customer for vehicle history and job history.
<code>pages/Leads.tsx</code>	Lead list with status management. Convert lead to customer inline.
<code>pages/Inbox.tsx</code>	WhatsApp conversation threads — one thread per lead or customer. Supports replying.
<code>pages/Settings.tsx</code>	Staff management (list, add, toggle active). Placeholder for pricing/notification settings.

Design Decision: The admin panel is deliberately a client-side SPA (not using React Server Components or Next.js Server Actions) because it needs to read the JWT from localStorage and make authenticated fetch calls. RSC runs on the server where localStorage is unavailable.

8. State Management & API Client

How the frontend talks to the backend.

The entire API surface is encapsulated in `src/lib/api.ts`. It exports:

- **auth** — localStorage token helpers (`setToken`, `getToken`, `clearToken`, `isLoggedIn`).
- **authApi** — `login()`, `me()`, `logout()`.
- **staffApi** — `list`, `get`, `create`, `update`.
- **dashboardApi** — `get()`.
- **jobsApi** — `list`, `get`, `create`, `update`, `advanceStatus`, `addNote`.
- **bookingsApi** — `list`, `create`, `update`, `confirm`, `delete`.
- **billsApi** — `list`, `get`, `create`, `update`, `send`, `recordPayment`, `getPublic`.
- **customersApi** — `list`, `lookup`, `get`, `create`, `update`, `getMessages`, `sendMessage`.
- **vehiclesApi** — `listByCustomer`, `create`, `update`, `delete`.
- **leadsApi** — `list`, `update`, `convert`, `getMessages`.
- **notifyApi** — `blast()`.

The core `api()` function:

- Reads the JWT from localStorage and attaches it as an Authorization: Bearer header.
- Sets Content-Type: application/json.
- Throws an object `{ error: string, status: number }` on non-2xx responses — the same shape as the backend error responses.
- Returns the parsed JSON body typed as `T`.

The `qs()` helper converts an object of query params (skipping undefined/empty values) into a URL query string.

Design Decision: All API calls go through one function. This means adding a feature like request retries, a loading interceptor, or a token refresh only requires a change in one place.

Design Tokens (`constants.ts`)

All colours, status display configurations, and navigation items are defined in `src/lib/constants.ts` and imported where needed. This avoids scattered magic strings:

- `c` — colour palette (`DARK`, `ORANGE`, `surface`, `text`, `border`, `success`, `warning`, etc.).
- `JOB_STATUS_CFG`, `BOOKING_STATUS_CFG`, `BILL_STATUS_CFG`, `LEAD_STATUS_CFG` — per-status label, background, text and dot colours used by badge components.
- `BOOKING_SOURCE_CFG` — colour coding for online/whatsapp/phone/walkin sources.
- `NAV` — the navigation item list with icon classes, ownerOnly flags, and page IDs.

9. Key Design Decisions — Summary Table

Quick reference for the 'why' behind every major choice.

Decision	Reason
Cloudflare Workers for backend	No server management, global edge deployment, generous free tier, one-command deploy. Trade-off: no native Node modules.
Hono framework	Built for the Workers runtime. Fastest router available on the edge. TypeScript-first with Zod support. Tiny bundle size.
Supabase (PostgreSQL)	Managed Postgres with Auth, RLS, JS client, and a web UI. No infra to manage. Free tier covers early stage. D1 can replace later for cost.
Custom JWT over Supabase JWT	Embeds role + location_id in the token payload. Avoids a DB lookup on every request. 7-day TTL matches typical admin session patterns.
Service-role Supabase client on backend	Backend owns all auth logic. RLS is bypassed in favour of application-level role checks in middleware. Simpler to reason about.
whatsapp_opt_in defaults to true	Customers explicitly want tracking updates. High WA open rates in India make it the right default. Users can opt out.
Public bill token (uuid)	Customer can view bill without login. Shareable link via WhatsApp. Token is opaque — no UUID enumeration possible with a random uuid.
Job status history is append-only	Complete audit trail. Can reconstruct the full timeline of any job. wa_sent flag shows which notifications fired.
Booking date filter in IST	Avoids the common UTC-midnight-bleed bug where a 11:30 PM IST booking shows up on tomorrow's list.
Zustand persist for booking wizard	Users often start a booking on mobile and complete later. Persisting state removes friction.
Admin is a client-side SPA	Auth requires localStorage access (server has none). React state naturally handles the multi-page admin without full-page reloads.
Revenue = amount_paid not total	Reflects real cash flow. total is what was invoiced; amount_paid is what was received. Unpaid invoices do not inflate revenue figures.
Sequential WA blasts (not parallel)	Meta's WhatsApp Business API enforces rate limits. Parallel sends would trigger 429 errors. Sequential sends stay within limits.
Phone as customer natural key	Phone is the primary identifier Indian customers use. Email is optional. Lookup-by-phone prevents duplicate records on job creation.

10. Data Flow Diagrams

The most important journeys through the system.

10.1 Online Booking Flow

- Step 1: Customer visits landing page.
- Step 2: Customer opens booking wizard (/booking). Selects vehicle → service → date.
- Step 3: Customer fills checkout form (/checkout): contact, address, problem.
- Step 4: If new customer, POST /public/otp/send fires a WhatsApp OTP to verify phone.
- Step 5: POST /public/otp/verify confirms the code.
- Step 6: POST /public/booking creates: customer (if new) → vehicle (if new) → booking (pending).
- Step 7: Owner receives a WhatsApp notification: 'New online booking from ...'.
- Step 8: Admin panel shows the booking in Bookings list (status: pending).
- Step 9: Owner confirms booking via admin → PATCH /bookings/:id/confirm.
- Step 10: Customer receives WhatsApp confirmation with slot date/time.
- Step 11: Vehicle arrives → owner creates a job from the booking (POST /jobs with booking_id). Booking status becomes 'converted'.

10.2 Job Lifecycle Flow

- Step 1: Job created (status = received). Status history entry logged.
- Step 2: Mechanic opens job in admin panel. Updates status to 'diagnosed'. Customer gets WA: 'Diagnosed — work begins shortly.'
- Step 3: Mechanic updates to 'in_progress'. Customer gets WA: 'Repairs are underway.'
- Step 4: Mechanic marks 'ready'. Customer gets WA: 'Ready for pickup!'
- Step 5: Owner creates bill (POST /bills). Adds labour + parts line items. Bill auto-calculates total.
- Step 6: Owner sends bill (POST /bills/:id/send). Customer gets WA link to public bill page.
- Step 7: Customer arrives. Owner records payment (PATCH /bills/:id/payment). Bill status → paid.
- Step 8: Owner marks job 'delivered'. Job disappears from active dashboard.

10.3 WhatsApp Lead Flow

- Step 1: Unknown number sends a WhatsApp message to the business WA number.
- Step 2: Meta sends POST to /webhook/whatsapp.
- Step 3: Signature is verified (HMAC-SHA256). Invalid signatures → 403.
- Step 4: Phone number is checked against customers table. Not found → lead is upserted.
- Step 5: Owner receives WA notification: 'New WhatsApp lead from +91...: message preview'.
- Step 6: Welcome template (rapidfix_welcome) is sent to the new contact.
- Step 7: Message is stored in messages table with lead_id.
- Step 8: Admin opens Inbox → selects lead → can read full conversation and reply.
- Step 9: When ready, admin converts lead to customer (POST /leads/:id/convert). All messages re-linked to customer_id.

10.4 Bill Payment Flow

- Step 1: Owner opens Jobs page, opens a job, scrolls to billing section.
- Step 2: POST /bills — creates draft bill linked to the job.
- Step 3: PATCH /bills/:id — adds line items (labour, parts). Server recalculates subtotal, total.
- Step 4: POST /bills/:id/send — marks bill as 'sent', delivers via WhatsApp. Customer receives a link.
- Step 5: Customer opens bill URL (no login required). Sees itemised invoice.
- Step 6: Vehicle delivered. Owner records payment: PATCH /bills/:id/payment with amount_paid.
- Step 7: If amount_paid >= total: status = 'paid'. If partial: 'partial'. Dashboard revenue updates.

11. Planned Features & Current Gaps

What is built, what is stubbed, what is next.

Built and Working

- Full job lifecycle management with WhatsApp notifications.
- Booking management with WhatsApp confirmation.
- Bill creation, delivery, and payment tracking with public bill link.
- Customer and vehicle database with history.
- Lead capture from website popup and inbound WhatsApp.
- Lead-to-customer conversion with message re-linking.
- Admin inbox for WhatsApp conversations.
- Role-based access (owner/admin/mechanic).
- Dashboard KPIs: revenue, job counts, leads, activity feed.
- OTP phone verification for public booking.
- Bulk WhatsApp blast to opted-in customers.
- Staff management.

Stubbed / Partially Built

- **Email delivery in bills:** The `POST /bills/:id/send` route has a `// TODO: send email via Resend` comment. WhatsApp delivery works; email does not yet.
- **Public booking server action:** The `src/app/actions/booking.ts` file still has a simulated delay instead of calling the real API. The public `/public/booking` endpoint is fully built on the backend — the frontend action needs to call it.
- **Payment gateway:** No payment integration exists. The bill currently records payments manually. Razorpay/Stripe integration is planned.
- **Pricing settings in admin:** The Settings page has a 'Coming Soon' placeholder for editing service pricing from the admin panel. Currently, pricing is hardcoded in the Zustand store.
- **WhatsApp opt-out handling:** If a customer sends 'STOP' to the WA number, Meta will stop delivering messages. The system does not yet process stop signals from the webhook to update `whatsapp_opt_in = false`.
- **SEO pages:** The `/mechanic-near-me-in-[city]` dynamic route exists for local SEO but content generation is not implemented.

Planned Next Steps (from project brief)

- **Bill + payment linking to dashboard:** Revenue figures already pull from bills. Once payment gateway is integrated, `amount_paid` will update automatically on payment webhook.
- **Marketing on customer database:** The `/notify/blast` endpoint is already built. The admin UI for composing and sending blasts needs to be added to the Settings or a new Marketing page.
- **Client-side database generation:** A reporting view of all customers with their vehicle history, spend, and last visit date — useful for re-engagement campaigns.

Environment Variables Reference

SUPABASE_URL	Supabase project URL. Non-secret (in wrangler.jsonc vars).
SUPABASE_SERVICE_KEY	Supabase service-role key. Bypasses RLS. Secret.
JWT_SECRET	HMAC secret for signing/verifying custom JWTs. Secret.
WA_VERIFY_TOKEN	Random string used to verify Meta webhook subscription. Secret.
WA_APP_SECRET	Meta app secret for HMAC-SHA256 webhook signature verification. Secret.
WA_NUMBER_ID	WhatsApp Business Phone Number ID from Meta dashboard.
WA_ACCESS_TOKEN	Meta Graph API access token (permanent token from system user). Secret.
OWNER_WA_NUMBER	Owner's personal WhatsApp number (digits only, e.g. 919876543210) for owner notifications.
FAST2SMS_API_KEY	SMS fallback key (not yet used in routes, reserved for future SMS support).
NEXT_PUBLIC_API_URL	Frontend env var: the deployed Worker URL e.g. https://repaiross-api.workers.dev